How the data connects

Graph technical reference — identity, resolution, every relationship, and the embedding layer on top.

22 node labels · 32 relationship types · 83 declarations over 96 files · 13,676,986 nodes / 16,830,561 relationships

1. The problem

Forty-nine sources, none of which were built to be joined. The FDA calls a drug `ATORVASTATIN CALCIUM`, Health Canada calls it `Atorvastatin calcium (as trihydrate)`, a trial registry writes `Lipitor 40mg`, and ChEMBL refers to `CHEMBL1487`. None of these strings match, and there is no identifier common to all four.

So the whole graph rests on one question: **given a name, which substance is it?** Everything else — products, trials, adverse events, papers — hangs off the answer. Sections 3 and 4 are that machinery; the rest is what gets built once it works.

Files, not a database

Every loader writes CSV. A validator reads those CSVs. Only a build that passes is imported. That ordering is the core of the design: a bulk import replaces the store outright with no transaction, so an unchecked build silently *becomes* the live graph, and the failure mode is a confident wrong answer rather than an error.

```
S3 (moine-data)
   |  77 declarations -> 96 CSV files
   v
build.py            stream, resolve names, emit nodes/edges
   v
~/graph-runs/<ts>/  nodes/*.csv  edges/*.csv  manifest.json
   v
validate.py         referential integrity, key collisions, fixtures
   v  (exit 0 required)
stage_for_neo4j.py  strip headers, collapse newlines, enforce types
   v
neo4j-admin import  -> Neo4j 'biolyt'
   v
schema.cypher       constraints, indexes, 3 full-text indexes
```

Nothing in the build talks to Neo4j. The CSVs a human can inspect are exactly the ones imported, so every claim about the graph is a claim about a table, checkable without infrastructure. The validator caught six structural bugs that way, including the key collision in section 2.

2. Identity: how a node gets its key

Every key is `NAMESPACE:VALUE` and globally unique — not unique per label, globally. `neo4j-admin import` resolves every endpoint in a single id space, so **two labels sharing a key become one node**, silently, with no error.

> **This happened.** `MESH:D000544` was the key of a Disease and also of an Identifier node recording that same MeSH id. On import they would have merged into a single node that was half disease, half identifier. The fix is that every Identifier is created through one function which prefixes its key with `ID:`, giving `ID:MESH:D000544`. Nothing else may create one.

namespace	label	value
`UNII:`	Substance	FDA UNII — the preferred key
`CHEMBL:`	Substance	molregno, when no UNII is known
`NAME:`	Substance	provisional — the name never resolved
`ATC:`	DrugClass	WHO ATC code, levels 1-5
`MESH:`	Disease	MeSH descriptor id
`UNIPROT: / CHEMBL_TARGET:`	Target	accession, else ChEMBL tid
`MECH:`	Mechanism	folded mechanism text
`NCT: EUCTR: CTIS: ...`	ClinicalTrial	registry id, 22 registries
`FDA: EMA: MHRA: CA: PMDA: SFDA:`	Product	agency's own product id
`APPROVAL: EVENT: PATENT: EXCL:`	Approval etc.	composite of agency and id
`COMPANY:`	Company	normalised company name
`COUNTRY: / REGION:`	Country / Region	ISO-3166 alpha-2, folded region
`AE: / SOC:`	AdverseEvent / OrganClass	MedDRA preferred term, organ class
`CLINVAR: / COSMIC:`	Variant	variation id
`PMID: / DOI:`	Publication	PubMed id, else DOI
`ID:`	Identifier	prefixed — see above

3. The resolver

One object, built during load 2 and shared by every loader afterwards. It answers `resolve(name) -> Match(key, method)` and **never returns nothing**.

3.1 Normalisation

function	does	example
`fold(s)`	lowercase, strip punctuation, collapse whitespace, hyphens to spaces	`ATORVASTATIN CALCIUM -> atorvastatin calcium`
`strip_salts(s)`	remove salt and hydrate suffixes	`atorvastatin calcium trihydrate -> atorvastatin`
`strip_stereo(s)`	remove stereochemical prefixes	`R-salbutamol -> salbutamol`
`norm_company(s)`	drop Inc, Ltd, GmbH, Pharmaceuticals	`Pfizer Inc. -> pfizer`

> `fold()` turning hyphens into spaces is load-bearing and has bitten once: a literal filter set containing `"0-unassigned"` never matched, because by the time the value reached it the string was `"0 unassigned"`. Fold both sides of any comparison.

3.2 The tiers

Four tables, tried in order. Separate tables rather than one, so a hit records *which* tier found it — that tier name is written onto every edge as `match_method`.

#	tier	matches on	method	confidence
1	`exact`	fold(name) -> UNII	`unii`	highest
2	`salt`	strip_salts(name) -> UNII	`salt`	high
3	`stereo`	strip_stereo(name) -> UNII	`stereo`	high, and guarded — see 3.3
4	`alias`	ChEMBL synonyms and brand names -> node key	`synonym`	good
—	`miss`	nothing matched	`provisional`	weak — see 3.4

Within each table the **first writer wins**. Loading order is therefore an authority ranking: gsrs preferred names, then gsrs synonyms, then ChEMBL. A gsrs name always beats a ChEMBL research code for the same string.

When two substances genuinely share a name, the first is kept and the collision is *recorded* rather than overwritten. Overwriting would make the result depend on file order.

3.3 Why the stereo tier is deferred

It cannot be built as names arrive, because two things must be true at once:

```
* The table must hold the PLAIN form. The prefix is usually on the
  QUERY, not the registered name: gsrs registers "Salbutamol",
  a source writes "R-Salbutamol". So salbutamol -> UNII must exist.

* That same entry must not let "Levo-cetirizine" reach cetirizine's
  UNII - they are different drugs. Whether it would depends on
  whether levocetirizine is separately registered, which is not
  known until every name has been loaded.
```

So `finalise()` proposes an entry for every name, then **withdraws** any whose stripped key could bridge two distinct substances. Conflict detection uses a looser pattern than the matcher, so run-together spellings (`levocetirizine`) are caught even though the strict matcher leaves them alone. Calling `resolve()` before `finalise()` triggers it automatically.

3.4 Provisional keys, and the merge that went wrong

A name that matches nothing still gets a node: `NAME:<folded>` , method `provisional` . Discarding the row instead would throw away a real product or trial because its ingredient string was unusual.

> **The bug this caused.** Provisional nodes were originally merged into ChEMBL molecules by name. ChEMBL contains descriptive names, so `NAME:platinum complex` matched 248 distinct molecules and absorbed them into one node; across the build, 22,125 provisional nodes swallowed 45,891 molecules. Merging is now conditional on the ChEMBL molecule having *resolved* to a real identifier.

Provisional nodes remain, correctly: they hold rows whose drug is genuinely unidentifiable from the text given. Treat any edge marked `provisional` as a hint.

4. Load order is a dependency chain

Order is not cosmetic. Each stage fills dictionaries the next one reads, and running them out of order does not fail — it silently produces fewer edges.

#	call	stage	why here
1	`load_atc`	**Vocabularies**	WHO ATC tree, 1,318 rows. Loaded whole even for a one-drug slice, because any substance may point into it.
2	`load_gsrs`	**Substance spine**	The naming authority. Builds the resolver every later loader matches names through, so nothing that needs a name can run before it.
3	`r.finalise()`	**Resolver finalised**	The stereo tier is built only once every name is known, so a key that two different substances both reduce to can be blocked rather than merged.
4	`load_chembl_molecules`	**ChEMBL molecules**	2.9M molecules, plus the molregno -> node-key map every later ChEMBL file needs.
5	`load_chembl_synonyms`	**ChEMBL synonyms**	Brand names and research codes, registered as the alias tier - consulted only after all three identifier tiers miss.

#	call	stage	why here
6	`load_structures`	**Structures**	InChIKey. Chemistry rather than a name, so the strongest merge signal available.
7	`reference.ALL`	**Reference tables**	ATC level 5, salt/parent hierarchy, RXCUI, SPL setids.
8	`load_targets`	**Targets**	Keyed by UniProt accession where one exists; the tid -> key map is held for mechanisms.
9	`load_mechanisms`	**Mechanisms**	One file yields both TARGETS and HAS_MECHANISM as stated fact.
10	`disease.ALL`	**Disease**	MeSH is the spine; chembl_indications carries the MeSH/EFO crosswalk that later loaders fold MONDO ids onto. Must precede anything matching disease prose.
11	`products.ALL`	**Products**	Ten agencies. Also creates all 189 Country and 9 Region nodes.
12	`trials.ALL`	**Trials**	Nine registries, WHO last so native records win on properties.
13	`safety.ALL`	**Safety**	Regulatory events, FAERS aggregation, VigiAccess organ classes.
14	`variants.ALL`	**Variants**	Needs symbol_target from HGNC and efo_mesh from chembl_indications.
15	`literature.ALL`	**Literature**	Needs a finalised resolver and mesh_by_name to match titles against.

5. Every relationship

All 32 of them. This table is asserted complete against the code at generation time, so a relationship added without a description here fails the build of this document.

relationship	from	to	match_method	what it means
`ABOUT`	Publication	Disease	`name`	Paper to disease, matched on TITLE ONLY.
`APPROVED_BY`	Product	RegulatoryAgency	`derived`	Which agency's register the product came out of. Derived rather than read - the file's location is the fact.
`APPROVED_IN`	Product	Region	`derived`	The agency's region, one hop pre-computed.
`ASSOCIATED_WITH`	Target	Disease	`structured`	Open Targets association, carrying their score.
`BIOSIMILAR_OF`	Product	Product	`structured`	Purple Book BLA reference relationship.
`CONDUCTED_IN`	ClinicalTrial	Country	`name`	Location, matched from free-text country names.

relationship	from	to	match_method	what it means
`CONTAINS`	Product	Substance	`resolver`	The active ingredient of a marketed product, matched from the ingredient string each agency prints. Carries the resolver tier that found it, so a weak match is visible on the edge.
`DEVELOPS`	Company	Product	`structured`	Marketing authorisation holder, as stated by the agency.
`HAS_ADVERSE_EVENT`	Substance	AdverseEvent	`aggregated`	FAERS reports, counted rather than listed: report, serious and death counts on the edge.
`HAS_APPROVAL`	Product	Approval	`structured`	The dated authorisation event, kept separate from the product so a product can carry several.
`HAS_EXCLUSIVITY`	Product	Exclusivity	`structured`	Orange Book and Purple Book market exclusivity.
`HAS_IDENTIFIER`	any	Identifier	`structured`	Every external id an entity carries. The one edge type reachable from all 22 labels.
`HAS_MECHANISM`	Substance	Mechanism	`structured`	Mechanism of action as ChEMBL states it.
`HAS_MODALITY`	Substance	Modality	`structured`	Small molecule, antibody, protein. Eleven values.
`HAS_ROUTE`	Product	Route	`structured`	Administration route, normalised by fold().
`IMPLICATED_IN`	Variant	Disease	`structured`	Variant to condition, carrying clinical significance.
`INDICATED_FOR`	Substance	Disease	`structured`	Approved or investigated indication.
`IN_CLASS`	Substance | Product | DrugClass	DrugClass	`resolver | structured`	ATC classification. Runs from a Substance, from a Product where the agency classified the product rather than the ingredient (28,579 of these, mostly Health Canada and EMA), and from a DrugClass to its own parent in the ATC tree.
`IN_ORGAN_CLASS`	AdverseEvent	OrganClass	`structured`	MedDRA system organ class. Makes 'any cardiac event' one hop.
`IN_REGION`	Country	Region	`structured`	189 countries into 9 regions.
`ISSUED_BY`	Approval | RegulatoryEvent	RegulatoryAgency | Company	`structured | derived`	Who issued an approval, or who issued a recall.
`IS_SALT_OF`	Substance	Substance	`structured`	Salt to parent. The reason a salt keeps its own node instead of being folded away.
`MENTIONS`	Publication	Substance	`resolver`	Paper to drug, matched on TITLE ONLY.
`PROTECTED_BY`	Product	Patent	`structured`	Orange Book patent listings.

relationship	from	to	match_method	what it means
`SAME_STUDY_AS`	ClinicalTrial	ClinicalTrial	`structured`	The same study registered twice. Both nodes are kept.
`SPONSORED_BY`	ClinicalTrial	Company	`structured`	Trial sponsor, normalised by norm_company().
`STUDIES`	ClinicalTrial	Disease	`name | name_variant | vocab_alias | icd_name`	Condition studied, matched from the registry's free-text condition field. Four tiers, most reliable first: the condition as written is a MeSH heading or entry term; a rewriting reached MeSH (a stage qualifier stripped, a plural, 'cancer' for 'neoplasms'); NCIt or CDISC lists it as a synonym of a concept that reaches MeSH; or only an ICD title matched.
`SUBJECT_OF`	Substance	RegulatoryEvent	`resolver`	Recall, shortage or safety communication naming the drug.
`SUBTYPE_OF`	Disease	Disease	`structured`	MeSH tree hierarchy.
`TARGETS`	Substance	Target	`symbol`	Drug to protein, from ChEMBL mechanisms and Open Targets, joined on gene symbol.
`TESTED_IN`	Substance	ClinicalTrial	`resolver`	Intervention, matched from the registry's free-text intervention field.
`VARIANT_IN`	Variant	Target	`symbol`	ClinVar and COSMIC variants to the gene they sit in.

5.1 What match_method is worth

Every edge carries it. It is the single most useful property for judging whether to trust an answer.

value	meaning	trust
`structured`	the source stated the relationship outright	**high**
`unii / salt / stereo`	resolved through an identifier tier	**high**
`symbol`	joined on a gene symbol	good
`derived`	computed from the file's own location or agency	high — but it is our inference, not the source's
`aggregated`	counted across many reports, not one fact	high in aggregate, meaningless per-report
`synonym`	matched via a brand name or research code	good
`ncit_oncology`	matched via an NCIt antineoplastic synonym	good
`name`	free prose matched against a dictionary	**hint only**

value	meaning	trust
`name_variant`	a rewriting of the prose reached the dictionary	**weaker than name**
`vocab_alias`	NCIt or CDISC names it as a synonym of a concept that reaches MeSH	**weaker than name**
`icd_name`	no MeSH form matched, an ICD title did	**weakest**
`provisional`	the name never resolved	**weak**

> `CONDUCTED_IN`, `STUDIES` and `ABOUT` rest entirely on matching text — **not one trial-to-disease edge in this graph is structured**. About 84% are exact `name` matches and the rest come from the three weaker tiers. They are sound for aggregate questions (*how many trials in the Gulf*) and should not be cited as fact about one specific trial.
>
> Measured against ClinicalTrials.gov itself, over 15 conditions and restricted to the trials taken from that registry, recall is **75%** — so a count from this graph is a floor, not a total.

6. The joins in detail

6.1 Products, and the identifier that did not join

Ten agency registers, each with its own product id, each naming ingredients as free text. Every product's ingredient string goes through the resolver; the tier that succeeds becomes `CONTAINS.match_method`. That single join is what makes *where is this drug approved* answerable across ten registers that share no key.

> **DailyMed's RXCUI looked like a free join and was not.** Both DailyMed and RxNorm publish RXCUI, so joining on it seemed obvious. Overlap was zero: DailyMed publishes *product-level* RXCUIs (SCD - a specific clinical drug, "atorvastatin 40 MG oral tablet") while the substance side carries *ingredient-level* ones (IN). Same column name, same id space, different granularity — and the symptom was not an error but an empty result. It now joins on `rxstring` through the resolver.

6.2 Trials across 22 registries

Registry ids are canonicalised to `NAMESPACE:VALUE`. The values look doubled — `NCT:NCT01045135`, `CTIS:CTIS2024-511` — because 19 of the 22 registries embed their own prefix in the id itself. Stripping it would make the key ambiguous against registries that do not.

WHO ICTRP mirrors other registries. It is loaded **last**, and merges into the native node rather than creating its own, so the native record wins on every property. Where two registries hold genuinely separate registrations of one study, both nodes are kept and joined by `SAME_STUDY_AS` — collapsing them would lose whichever fields only one side recorded.

6.3 Disease, and the crosswalk

MeSH is the spine. Other sources speak EFO, MONDO or plain prose. `chembl_indications` carries a MeSH/EFO pair per row, and that is the crosswalk every later loader folds its own ids onto — which is why disease must load before anything that matches disease text.

Free-text conditions from trial registries are matched against the MeSH name and synonym dictionary. Synonyms matter more than names here: the descriptor is *Carcinoma, Non-Small-Cell Lung* and no human writes that.

6.4 Safety: counted, not listed

FAERS is tens of millions of individual reports. Storing one edge per report would add more relationships than the rest of the graph holds and answer no question better. Reports are aggregated per substance-reaction pair, with report, serious and death counts on the edge, method `aggregated`.

`IN_ORGAN_CLASS` then hangs each reaction under a MedDRA system organ class, so *any cardiac event* is one hop rather than an enumeration of every cardiac term.

6.5 Variants

ClinVar is ~21.8M rows and **has no header** — the first line is data, so columns are read by position. Only the first 34 of 43 are positionally stable across releases. Filtered hard on load: the gene must be a known drug target and the clinical significance must be an actual call, not *uncertain* or *conflicting*. 937,377 survive.

This is what makes mutation → protein → drug a traversal: `VARIANT_IN` joins on gene symbol, `TARGETS` already joins drugs to those proteins.

6.6 Literature: title only

Publications are matched to diseases and drugs on their **title only, never the abstract**. An abstract mentions every drug in a comparison, every disease in a differential, and every compound in a screen. Matching on abstracts produces enormous numbers of edges that each mean nothing more than 'this word appeared'. The title is a claim about what the paper is about.

6.7 Provenance on every row

Every node and edge carries which source file produced it. Sources are written as short ids resolved through a side table — full S3 keys repeated across 30M rows cost 2.2 GB of pure repetition.

7. Validation

Exit non-zero here and no import happens. Checks:

```
referential integrity  every edge endpoint exists as a node
key uniqueness         no key used by two labels (section 2)
resolution quality     provisional share within bounds per source
fan-out outliers       a node absorbing implausibly many edges
source coverage        every declared file actually read
biology fixtures       atorvastatin -> HMG-CoA reductase
                       pembrolizumab -> PD-1
                       erenumab -> CGRP receptor
```

Fan-out is what caught the platinum-complex merge; source coverage caught five declared files that were never loaded. Fixtures are skipped on a `--slice` build, since pembrolizumab cannot pass on an atorvastatin slice, and enforced on every full build.

> The validator holds **keys and counters, not rows**. It once held every row as a dict: 11.5M of them beside Neo4j's 8 GB drove the host into swap and dropped the SSH session mid-import.

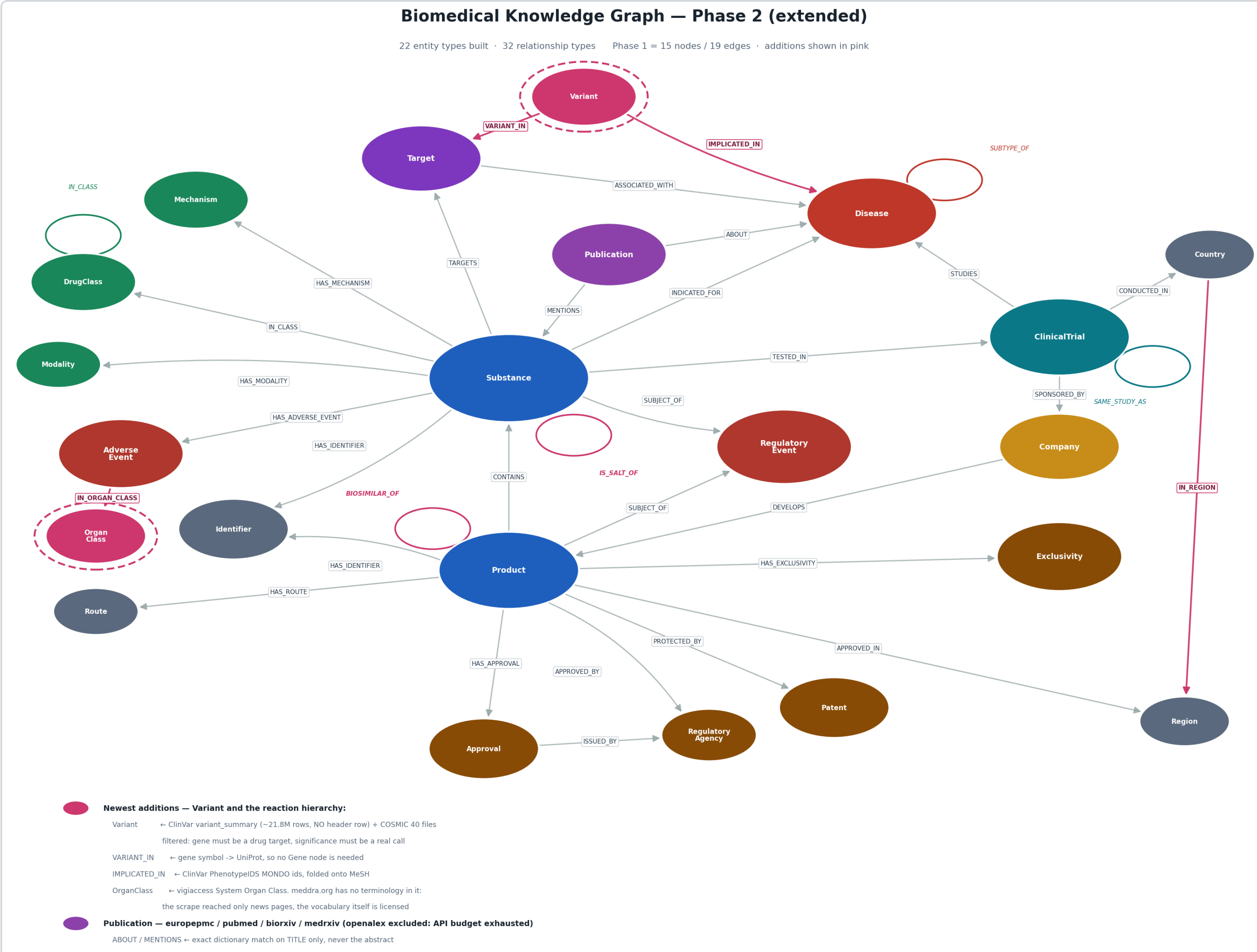
8. Import

`stage_for_neo4j.py` runs first: strips header rows, collapses embedded newlines, and blanks values that do not match their declared type. It exists because three separate imports failed on it — the last, an `enrollment` column where 131,158 ChiCTR rows hold prose rather than an integer.

Types are declared in headers because untyped means string, and a string comparison is silent: without `:int` on `report_count`, `WHERE e.report_count > 100` sorts `9` above `100` and raises nothing.

> Import runs with `--bad-tolerance=0`. The validator has already proved every endpoint resolves, so a bad relationship at this point means the files changed between validating and importing — and a partial import that drops edges silently is worse than a failed one. The store is dumped first; that dump is the only way back.

9. Schema



label	#	properties (before provenance)
`AdverseEvent`	2	`key` , `term`
`Approval`	5	`key` , `date` , `type` , `status` , `agency`
`ClinicalTrial`	9	`key` , `registry` , `title` , `status` , `phase` , `study_type` , `study_type_raw` , `enrollment` , `start_date`
`Company`	3	`key` , `name` , `raw_names`
`Country`	3	`key` , `iso2` , `name`
`Disease`	5	`key` , `name` , `synonyms` , `vocabulary` , `tree_numbers`
`DrugClass`	5	`key` , `atc_code` , `name` , `level` , `vocabulary`
`Exclusivity`	4	`key` , `code` , `date` , `definition`
`Identifier`	3	`key` , `scheme` , `value`
`Mechanism`	3	`key` , `name` , `action_type`

label	#	properties (before provenance)
`Modality`	2	`key` , `name`
`OrganClass`	2	`key` , `name`
`Patent`	7	`key` , `patent_no` , `expire_date` , `use_code` , `use_definition` , `drug_substance_flag` , `drug_product_flag`
`Product`	7	`key` , `name` , `agency` , `status` , `status_raw` , `form` , `strength`
`Publication`	8	`key` , `title` , `year` , `journal` , `doi` , `pmid` , `source_db` , `preprint`
`Region`	2	`key` , `name`
`RegulatoryAgency`	5	`key` , `code` , `name` , `country` , `region`
`RegulatoryEvent`	8	`key` , `type` , `name` , `status` , `reason` , `start_date` , `end_date` , `url`
`Route`	2	`key` , `name`
`Substance`	8	`key` , `name` , `norm_name` , `synonyms` , `substance_class` , `status` , `max_phase` , `resolved_by`
`Target`	5	`key` , `symbol` , `name` , `organism` , `target_type`
`Variant`	7	`key` , `name` , `variant_type` , `gene_symbol` , `clinical_significance` , `catalogue` , `consequence`

10. Embedding

The last layer, and the one that decides whether a question finds its starting node at all. Everything above connects nodes to each other; this connects *a question* to a node.

10.1 Why full-text was not enough

Three full-text indexes cover names, titles and reaction terms, and they handle exact and near-exact strings well. They cannot bridge a query to a name it shares no characters with:

query	full-text	the node it should find	embedding score
`NSCLC`	*nothing*	Non-small cell lung cancer	**0.832**
`statins`	*nothing*	HMG-CoA reductase inhibitor	**0.737**
`COPD`	*nothing*	Chronic obstructive pulmonary disease	**0.803**
`heart problems`	*nothing*	heart disorder	**0.873**

10.2 Which model, and how that was decided

Measured, not assumed. **SapBERT** (`cambridgeltl/SapBERT-from-PubMedBERT-fulltext` , 768-d, [CLS] pooling) was compared against **bge-m3** (1024-d, already in use for documents) on the same probes.

	SapBERT	bge-m3
probes correct	**6 / 6**	5 / 6
score range on correct answers	**0.74 - 0.87**	0.43 - 0.61

The score range matters more than the hit count. SapBERT is trained on biomedical entity linking — on exactly this task, aligning a surface form to a concept — so its correct answers sit far from its wrong ones and a threshold is meaningful. bge-m3's correct answers score close to its incorrect ones, which leaves no way to reject a bad match.

> The two models coexist on purpose. Documents stay on **bge-m3** in Qdrant (1024-d): passage retrieval is a different task from entity linking, and re-embedding 3.24M chunks to unify them would cost hours of GPU to make retrieval worse.

10.3 What is embedded, and what is not

label	nodes	why
`Disease`	24,488	questions arrive as acronyms and lay phrasing
`DrugClass`	6,996	'statins' is a class nobody spells as an ATC name
`AdverseEvent`	6,981	MedDRA terms are clinical; complaints are not
`Mechanism`	1,967	described in prose that varies freely

40,432 nodes total. Substance, Product, Target, Company and ClinicalTrial are deliberately excluded: those are matched by identifier or exact name, where full-text already wins and an embedding would introduce plausible wrong answers. Embedding 3.07M substances would also cost far more than it returns — 93% of them have no name at all.

10.4 Using it

```
python graph/embed_entities.py --dir ~/graph-runs/<ts> \
                               --query "NSCLC"
```

> **Reject matches below 0.60.** Correct answers measured 0.74-0.87 and the best wrong answer sat well below 0.60, so the gap is real. Use it — an embedding always returns a nearest neighbour, and without a floor the nearest neighbour to a nonsense query is presented with the same confidence as a correct one.

Order of preference when finding a starting node: an exact identifier if you have one, then full-text with a label filter, then embedding for acronyms and paraphrase. Full detail in `graph/AGENT_QUERY_GUIDE.md`.